# CocoLuxe Punta Cana

## SOLUCIÓN TÉCNICA v2 — TRACKING DEL CLIENTE + ENCUESTA POST-VIAJE

Sin HyperTrack · Sin App · Solo el navegador del cliente · Supabase Realtime · Google Maps · Make

Caribbean Matrix SRL · Para el Desarrollador · Documento Oficial · Abril 2026

| ✕ LO QUE NO SE USA | ✓ LO QUE SE USA |
| --- | --- |
| **HyperTrack** | **Navegador + Supabase** |
| Requiere app nativa instalada.Diseñado para rastrear vehículos/choferes.No para la experiencia del cliente. | navigator.geolocation nativo.Supabase Realtime para distribuir.Google Maps para visualizar.Costo adicional: $0. |

# 1. Contexto Operativo — Cómo Funciona el Negocio

Antes de implementar cualquier solución técnica, el desarrollador debe entender cómo opera CocoLuxe realmente. Esto determina qué campos son opcionales, cuándo se muestran y qué lógica condicional necesita la página del cliente.

## 1.1 — La Realidad del Negocio

- **CocoLuxe subcontrata choferes externos — no siempre son de la flota propia.**
- Cuando el cliente reserva, en muchos casos el vehículo y la placa NO están definidos todavía.
- El vehículo se asigna horas o incluso el mismo día del servicio — cuando se consigue el chofer.
- El cliente siempre reserva con anticipación — el sistema debe funcionar sin tener el vehículo asignado.
- Hay tres tipos de chofer: propio con flota propia, externo conocido frecuente, externo desconocido contratado puntualmente.

> **PRINCIPIO DE DISEÑO:** *La página del cliente nunca muestra datos vacíos ni "TBD" ni "N/A". Si el vehículo no está asignado, muestra un mensaje claro y elegante que comunica que la asignación viene pronto. Cuando se asigna, la página se actualiza automáticamente sin que el cliente haga nada.*

## 1.2 — Lo Que el Cliente SIEMPRE ve (desde el momento de reservar)

| DATO | FUENTE | SIEMPRE DISPONIBLE |
|---|---|---|
| Número de confirmación | Generado al crear la reserva | ✅ Siempre |
| Fecha del servicio | Ingresado por el cliente al reservar | ✅ Siempre |
| Hora de recogida | Auto-llenada por AviationStack o manual | ✅ Siempre |
| Tipo de servicio | Transfer / Tour / Rental | ✅ Siempre |
| Origen | Aeropuerto PUJ / Hotel / Dirección | ✅ Siempre |
| Destino | Hotel / Destino del tour / Airbnb | ✅ Siempre |
| Contacto CocoLuxe | WhatsApp + email corporativo | ✅ Siempre |
| Estado de la reserva | Confirmed / In Progress / Completed | ✅ Siempre |

## 1.3 — Lo Que el Cliente ve SOLO cuando está asignado

| DATO | FUENTE | CUÁNDO APARECE |
|---|---|---|
| Tipo de vehículo | Asignado en el dashboard interno | Solo cuando el campo vehicle en bookings tiene valor |
| Placa del vehículo | Asignado en el dashboard interno | Solo cuando el campo plate en bookings tiene valor |
| Botón "Iniciar mi viaje" | Activado al asignar vehículo | Solo cuando vehículo Y placa están asignados |
| Nombre del chofer (opcional) | Asignado en el dashboard | Solo si CocoLuxe decide mostrarlo — campo opcional |

> ⚠ **IMPORTANTE: El botón "Iniciar mi viaje" NUNCA aparece si el vehículo no está asignado. No tiene sentido que el cliente active un tracking si no sabe qué unidad lo va a buscar. El botón es la confirmación visual de que todo está listo.**

# 2. Página del Cliente — /reserva/[id]

Esta es la página más importante del proyecto desde la perspectiva del cliente. Es lo primero que ve cuando recibe la confirmación de su reserva. Debe funcionar perfectamente en cualquier teléfono del mundo, en el idioma en que el cliente reservó, y actualizarse automáticamente cuando CocoLuxe asigne el vehículo.

## 2.1 — Los 4 Estados de la Página

La página tiene 4 estados distintos. El desarrollador debe implementar los 4 con lógica condicional basada en los campos de la reserva en Supabase.

### ESTADO 1 — Reserva Confirmada (sin vehículo asignado)

**CUÁNDO:** Inmediatamente después de reservar. El vehículo y la placa no están definidos todavía.
**EL CLIENTE VE:** Número de confirmación · Fecha · Hora · Tipo de servicio · Origen · Destino · Mensaje elegante de asignación pendiente · WhatsApp CocoLuxe · SIN botón de tracking

### ESTADO 2 — Vehículo Asignado

**CUÁNDO:** CocoLuxe asignó el vehículo en el dashboard. Supabase Realtime actualiza la página automáticamente.
**EL CLIENTE VE:** Todo del Estado 1 + Tipo de vehículo + Placa + Notificación push/badge "Your vehicle is ready" + Botón "Iniciar mi viaje" (activo)

### ESTADO 3 — Tracking Activo

**CUÁNDO:** El cliente tocó "Iniciar mi viaje" y aceptó el permiso de GPS.
**EL CLIENTE VE:** Mapa de Google Maps con punto dorado moviéndose · Tiempo estimado de llegada · Botón "Share Journey" · Botón "Llegué"

### ESTADO 4 — Viaje Completado

**CUÁNDO:** El cliente tocó "Llegué" o el sistema detectó proximidad al destino.
**EL CLIENTE VE:** Mensaje de bienvenida · Posición final en el mapa estática · "Thank you for traveling with CocoLuxe" · Encuesta llegará en 2 horas

## 2.2 — El Mensaje de Vehículo Pendiente (Estado 1)

Este es el mensaje que ve el cliente cuando el vehículo no está asignado todavía. Debe verse premium, no como un error.

| EN (inglés) | ES (español) |
|---|---|
| *"Your vehicle will be confirmedbefore your service date.You will receive a notificationwith all details."* | *"Tu vehículo será confirmadoantes de la fecha de tu servicio.Recibirás una notificacióncon todos los detalles."* |

| FR (francés) | PT (portugués) |
|---|---|

| | |
|---|---|
| *"Votre véhicule sera confirméavant votre date de service.Vous recevrez une notificationavec tous les détails."* | *"Seu veículo será confirmadoantes da data do serviço.Você receberá uma notificaçãocom todos os detalhes."* |

# 3. Código Completo — Página del Cliente con Lógica Condicional

Crear el archivo: src/app/[locale]/reserva/[id]/page.jsx
Este código maneja los 4 estados automáticamente según los datos en Supabase.

## 3.1 — Código Completo de la Página

```
'use client';
import { useState, useEffect, useRef } from 'react';
import { createClient } from '@supabase/supabase-js';
import LiveMap from '@/components/LiveMap';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
);

export default function BookingPage({ params }) {
  const { id, locale } = params;

  // ── ESTADO ──
  const [booking, setBooking]     = useState(null);
  const [tracking, setTracking]   = useState(false);
  const [position, setPosition]   = useState(null);
  const [shareUrl, setShareUrl]   = useState(null);
  const [ended, setEnded]         = useState(false);
  const [loading, setLoading]     = useState(true);
  const [trackId, setTrackId]     = useState(null);
  const watchRef                  = useRef(null);

  // ── CARGAR RESERVA Y ESCUCHAR CAMBIOS EN TIEMPO REAL ──
  useEffect(() => {
    // Carga inicial
    supabase.from('bookings').select('*').eq('id', id)
      .single().then(({ data }) => { setBooking(data); setLoading(false); });

    // Supabase Realtime — se actualiza automáticamente cuando
    // CocoLuxe asigna el vehículo desde el dashboard
    const channel = supabase
      .channel('booking-' + id)
      .on('postgres_changes', {
        event: 'UPDATE',
        schema: 'public',
        table: 'bookings',
        filter: 'id=eq.' + id
      }, (payload) => {
        // La página se actualiza sola — el cliente no recarga
        setBooking(payload.new);
      })
      .subscribe();

    return () => supabase.removeChannel(channel);
```

```
}, [id]);

// ── LÓGICA CONDICIONAL — QUÉ ESTADO ESTÁ ACTIVO ──
// Estado 1: sin vehículo asignado
const vehicleAssigned = booking?.vehicle && booking?.plate;
// Estado 3: tracking activo
const isTracking = tracking && position;
// Estado 4: viaje terminado
const isEnded = ended;

// ── INICIAR TRACKING (Estado 2 → Estado 3) ──
const startTracking = async () => {
  if (!navigator.geolocation) {
    alert(t('gps_not_supported'));
    return;
  }

  // Crear registro en live_tracking
  const { data } = await supabase
    .from('live_tracking')
    .insert({ booking_id: id, latitude: 0, longitude: 0, is_active: true })
    .select().single();
  setTrackId(data.id);
  setShareUrl(window.location.origin + '/seguimiento/' + id);
  setTracking(true);

  // GPS del navegador — actualiza cada 5 segundos
  watchRef.current = navigator.geolocation.watchPosition(
    async (pos) => {
      const coords = {
        latitude:  pos.coords.latitude,
        longitude: pos.coords.longitude,
        accuracy:  pos.coords.accuracy,
        speed:     pos.coords.speed || 0
      };
      setPosition({ lat: coords.latitude, lng: coords.longitude });
      // Supabase Realtime distribuye a quien tenga el enlace de seguimiento
      await supabase.from('live_tracking')
        .update(coords).eq('id', data.id);
    },
    (err) => console.error('GPS:', err),
    { enableHighAccuracy: true, maximumAge: 5000, timeout: 10000 }
  );
};

// ── TERMINAR VIAJE (Estado 3 → Estado 4) ──
const endTrip = async () => {
  if (watchRef.current)
    navigator.geolocation.clearWatch(watchRef.current);

  // Marcar como inactivo — el enlace de seguimiento se congela
  await supabase.from('live_tracking')
    .update({ is_active: false, ended_at: new Date().toISOString() })
    .eq('id', trackId);
```

```
  // Notificar a Make → Make espera 2h → envía encuesta
  await fetch('/api/trips/complete', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ bookingId: id })
  });

  setEnded(true);
  setTracking(false);
};

// ── COMPARTIR VIAJE ──
const shareJourney = () => {
  if (navigator.share) {
    navigator.share({
      title: 'My CocoLuxe Journey',
      url: shareUrl
    });
  } else {
    navigator.clipboard.writeText(shareUrl);
    alert('Link copied to clipboard!');
  }
};

// ── RENDER ──
if (loading) return <LoadingScreen />;
if (!booking) return <NotFoundScreen />;

return (
  <main className='booking-page'>

    {/* HEADER */}
    <Header />

    {/* NÚMERO DE CONFIRMACIÓN — siempre visible */}
    <ConfirmationNumber id={booking.id} />

    {/* TARJETA DE DETALLES DEL SERVICIO — siempre visible */}
    <ServiceCard booking={booking} locale={locale} />

    {/* TARJETA DEL VEHÍCULO — condicional */}
    {vehicleAssigned ? (
      <VehicleCard vehicle={booking.vehicle} plate={booking.plate} />
    ) : (
      <VehiclePending locale={locale} />
    )}

    {/* MAPA — solo cuando el tracking está activo */}
    {isTracking && <LiveMap position={position} />}

    {/* ESTADO 4: VIAJE TERMINADO */}
    {isEnded && <TripEndedCard locale={locale} />}

    {/* BOTONES — condicionales según el estado */}
    {!isEnded && !tracking && vehicleAssigned && (
```

```
        <button className='btn-primary' onClick={startTracking}>
          {t('start_journey', locale)}
        </button>
      )}
      {tracking && !isEnded && (
        <>
          <button className='btn-secondary' onClick={shareJourney}>
            {t('share_journey', locale)}
          </button>
          <button className='btn-outline' onClick={endTrip}>
            {t('i_arrived', locale)}
          </button>
        </>
      )}

      {/* FOOTER — siempre visible */}
      <BookingFooter />
    </main>
  );
}
```

# 4. Subcomponentes — Implementación de Cada Tarjeta

Cada sección de la página es un componente separado en src/components/Booking/. Esto facilita el mantenimiento y el testing individual de cada parte.

## 4.1 — ServiceCard — Tarjeta de Detalles del Servicio

Siempre visible. Muestra todos los datos confirmados de la reserva.

| CAMPO | DATO DE SUPABASE | FORMATO A MOSTRAR |
|---|---|---|
| Número de confirmación | bookings.id | CL-48291 — grande, dorado, centrado |
| Tipo de servicio | bookings.type | ✈ Airport Transfer / 🌴 Tour / 🚗 Rental |
| Fecha | bookings.date | Martes, 15 de abril de 2026 (localizado según idioma) |
| Hora de recogida | bookings.time | 3:30 PM — formato 12h para turistas |
| Origen | bookings.from_location | Punta Cana International Airport (PUJ) |
| Destino | bookings.to_location | Excellence El Carmen, Uvero Alto |
| Pasajeros | bookings.adults + bookings.children | 2 adults · 1 child |
| Número de vuelo | bookings.flight_number | ✈ AA3209 — con indicador de tracking activo |

## 4.2 — VehicleCard y VehiclePending — Dos Versiones

El mismo espacio en la pantalla muestra dos cosas distintas según si el vehículo fue asignado o no.

### VehicleCard — cuando SÍ está asignado

- Ícono del tipo de vehículo (🚙 SUV, 🚐 Van, 🚌 Coach)
- Nombre del vehículo — "Chevrolet Suburban" / "Toyota Hiace"
- Placa — grande y legible — "ABC-001"
- Nombre del chofer (opcional — solo si CocoLuxe decide mostrarlo)
- Badge dorado: "Your vehicle is confirmed ☑"

### VehiclePending — cuando NO está asignado todavía

- Ícono de reloj o calendario — no un error
- Texto en el idioma del cliente (ver sección 2.2 para los 4 textos)
- Badge naranja sutil: "Assignment pending"
- Cuando Supabase Realtime actualice el campo, este componente se reemplaza automáticamente por VehicleCard — sin recargar la página

## 4.3 — Notificación al Cliente Cuando se Asigna el Vehículo

Cuando CocoLuxe asigna el vehículo desde el dashboard, Make envía automáticamente este mensaje al cliente:

📱 **SMS/WhatsApp que recibe el cliente**

CocoLuxe: Your vehicle for tomorrow's transfer has been confirmed.

🚙 **Chevrolet Suburban · Plate: ABC-001**

📅 April 15 at 3:30 PM · PUJ → Excellence El Carmen

*View your booking: cocoluxepuntacana.com/reserva/CL-48291*

> **FLUJO EN MAKE:** *El disparador de esta notificación es una actualización en Supabase — cuando el campo vehicle de la tabla bookings pasa de NULL a tener un valor. Make detecta ese cambio y envía el SMS/email automáticamente. El desarrollador debe configurar este webhook de Supabase → Make como parte del flujo de automatizaciones.*

# 5. Tracking — Implementación Completa

## 5.1 — Tabla Supabase: live_tracking

Ejecutar en Supabase → SQL Editor → New Query:

```
CREATE TABLE live_tracking (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  booking_id TEXT REFERENCES bookings(id) ON DELETE CASCADE,
  latitude DECIMAL(10,8) NOT NULL DEFAULT 0,
  longitude DECIMAL(11,8) NOT NULL DEFAULT 0,
  accuracy DECIMAL(8,2),
  speed DECIMAL(6,2) DEFAULT 0,
  is_active BOOLEAN DEFAULT TRUE,
  started_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  ended_at TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Activar Realtime para distribución instantánea
ALTER TABLE live_tracking REPLICA IDENTITY FULL;

-- Seguridad: solo el backend escribe, lectura pública solo si activo
ALTER TABLE live_tracking ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Read active tracking"
  ON live_tracking FOR SELECT
  USING (is_active = TRUE);
```

## 5.2 — Componente LiveMap con Estilos CocoLuxe

Crear: src/components/LiveMap.jsx

```
import { GoogleMap, Marker, useJsApiLoader } from '@react-google-maps/api';

// Estilos del mapa en paleta negro/dorado de CocoLuxe
const mapStyles = [
  { elementType: 'geometry',
    stylers: [{ color: '#0d1b2a' }] },
  { elementType: 'labels.text.fill',
    stylers: [{ color: '#C9A84C' }] },
  { elementType: 'labels.text.stroke',
    stylers: [{ color: '#0A0A0A' }] },
  { featureType: 'road', elementType: 'geometry',
    stylers: [{ color: '#1a2a3a' }] },
  { featureType: 'road.highway', elementType: 'geometry',
    stylers: [{ color: '#2a3a4a' }] },
  { featureType: 'water', elementType: 'geometry',
    stylers: [{ color: '#0a0a0a' }] },
  { featureType: 'poi',
    stylers: [{ visibility: 'off' }] },
  { featureType: 'transit',
```

```
    stylers: [{ visibility: 'off' }] }
];

export default function LiveMap({ position }) {
  const { isLoaded } = useJsApiLoader({
    googleMapsApiKey: process.env.NEXT_PUBLIC_GOOGLE_MAPS_KEY
  });

  if (!isLoaded) return (
    <div style={{ background:'#0D1B2A', height:'360px',
      borderRadius:'12px', display:'flex',
      alignItems:'center', justifyContent:'center',
      color:'#C9A84C' }}>
      Loading map...
    </div>
  );

  if (!position) return (
    <div style={{ background:'#0D1B2A', height:'360px',
      borderRadius:'12px', display:'flex',
      alignItems:'center', justifyContent:'center',
      color:'#888' }}>
      Waiting for GPS signal...
    </div>
  );

  return (
    <GoogleMap
      mapContainerStyle={{
        width: '100%', height: '360px', borderRadius: '12px'
      }}
      center={position}
      zoom={15}
      options={{
        styles: mapStyles,
        disableDefaultUI: true,
        zoomControl: true,
        gestureHandling: 'greedy'
      }}
    >
      <Marker
        position={position}
        icon={{
          path: 'M12 0C7.6 0 4 3.6 4 8c0 5.5 8 13 8 13s8-7.5 8-13c0-4.4-3.6-8-8-8z',
          fillColor: '#C9A84C',
          fillOpacity: 1,
          strokeColor: '#0A0A0A',
          strokeWeight: 2,
          scale: 1.8
        }}
      />
    </GoogleMap>
  );
}
```

## 5.3 — Página de Seguimiento (Solo Lectura)

Crear: src/app/[locale]/seguimiento/[id]/page.jsx

Esta página la ve quien recibe el enlace compartido. Solo el mapa — sin controles.

```
'use client';
import { useEffect, useState } from 'react';
import { createClient } from '@supabase/supabase-js';
import LiveMap from '@/components/LiveMap';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
);

export default function TrackingView({ params }) {
  const { id } = params;
  const [position, setPosition] = useState(null);
  const [booking, setBooking]   = useState(null);
  const [active, setActive]     = useState(true);

  useEffect(() => {
    // Cargar datos de la reserva para mostrar origen/destino
    supabase.from('bookings').select('from_location,to_location,type')
      .eq('id', id).single()
      .then(({ data }) => setBooking(data));

    // Escuchar actualizaciones de posición en tiempo real
    const ch = supabase.channel('tracking-view-' + id)
      .on('postgres_changes', {
        event: 'UPDATE', schema: 'public',
        table: 'live_tracking',
        filter: 'booking_id=eq.' + id
      }, ({ new: row }) => {
        if (!row.is_active) { setActive(false); return; }
        setPosition({ lat: row.latitude, lng: row.longitude });
      })
      .subscribe();

    return () => supabase.removeChannel(ch);
  }, [id]);

  return (
    <main style={{ background:'#0A0A0A', minHeight:'100vh', color:'#fff' }}>
      <Header />
      {active ? (
        <>
          {booking && (
            <div style={{ textAlign:'center', padding:'16px' }}>
              <span style={{ color:'#C9A84C', fontWeight:700 }}>
                {booking.from_location}
              </span>
              {' → '}
              <span style={{ color:'#fff' }}>{booking.to_location}</span>
            </div>
```

```
          )}
          <LiveMap position={position} />
          <p style={{ textAlign:'center', color:'#888', marginTop:'12px' }}>
            Live tracking · Updates every 5 seconds
          </p>
        </>
      ) : (
        <div style={{ textAlign:'center', padding:'60px 20px' }}>
          <p style={{ color:'#C9A84C', fontSize:'48px' }}>✓</p>
          <p style={{ color:'#fff', fontSize:'20px' }}>
            Journey completed. Welcome!
          </p>
        </div>
      )}
    </main>
  );
}
```

# 6. Make — Flujo de Notificación al Asignar Vehículo

Este es un flujo adicional que se activa cuando CocoLuxe asigna el vehículo desde el dashboard. Es diferente al flujo de encuesta post-viaje.

## 6.1 — Cómo Configurar el Trigger en Supabase → Make

| | | |
|---|---|---|
| 1 | En Make | Crear nuevo escenario → Webhook → Custom Webhook → copiar URL |
| 2 | En Supabase | Database → Webhooks → Create webhook |
| 3 | Configurar | Table: bookings · Events: UPDATE · Filter: vehicle IS NOT NULL · URL: la URL de Make |
| 4 | En Make | Después del webhook → Supabase (get booking + client) → Resend email → Twilio SMS |
| 5 | Verificar | Actualizar el campo vehicle de una reserva de prueba → Make debe recibir el evento |

## 6.2 — Los 2 Flujos Relacionados con la Página del Cliente

| FLUJO | TRIGGER | ACCIÓN | RESULTADO PARA EL CLIENTE |
|---|---|---|---|
| Vehículo asignado | UPDATE en bookings donde vehicle cambia de NULL a valor | Resend email + Twilio SMS con datos del vehículo | La página se actualiza automáticamente y llega notificación |
| Viaje completado | POST a /api/trips/complete desde el botón "Llegué" | Make espera 2h → Resend email + SMS con encuesta | Cliente recibe las 3 opciones de review/encuesta |

# 7. Encuesta Post-Viaje — /feedback/[id]

Página construida directamente en Next.js. Sin Typeform. Sin servicios externos. Cero costo adicional.

## 7.1 — Las 3 Preguntas en los 4 Idiomas

| PREGUNTA | EN | ES | FR | PT |
|---|---|---|---|---|
| Calificación | How was your experience? | ¿Cómo fue tu experiencia? | Comment était votre expérience? | Como foi sua experiência? |
| Lo mejor | What did you love most? | ¿Qué fue lo mejor? | Qu'avez-vous aimé le plus? | O que você mais gostou? |
| Mejorar | Any suggestions for us? | ¿Algo que mejorar? | Des suggestions pour nous? | Alguma sugestão para nós? |

## 7.2 — Endpoint /api/feedback/route.js

```
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY
);

export async function POST(request) {
  const { bookingId, rating, comment, improvement } = await request.json();

  // Obtener client_id de la reserva
  const { data: booking } = await supabase
    .from('bookings').select('client_id')
    .eq('id', bookingId).single();

  // Guardar encuesta
  await supabase.from('feedback').insert({
    booking_id: bookingId,
    client_id: booking.client_id,
    rating, comment, improvement,
    source: 'internal'
  });

  // Notificar a Make: si rating <= 2 alertar a Kelmin
  await fetch(process.env.MAKE_WEBHOOK_FEEDBACK, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ bookingId, rating, comment })
  });

  return Response.json({ success: true });
}
```

# 8. Variables de Entorno Nuevas para Esta Solución

Agregar en Vercel → Settings → Environment Variables Y en .env.local para desarrollo:

| VARIABLE | DÓNDE OBTENERLA | PARA QUÉ SE USA |
|---|---|---|
| NEXT_PUBLIC_GOOGLE_MAPS_KEY | console.cloud.google.com → APIs → Credentials → Create API Key | Mostrar el mapa en la página del cliente y en seguimiento |
| MAKE_WEBHOOK_TRIP_COMPLETED | Make → nuevo escenario → Custom Webhook → copiar URL | Notificar a Make cuando el cliente toca "Llegué" |
| MAKE_WEBHOOK_FEEDBACK | Make → nuevo escenario → Custom Webhook → copiar URL | Notificar a Make cuando el cliente responde la encuesta |
| MAKE_WEBHOOK_VEHICLE_ASSIGNED | Make → nuevo escenario → Custom Webhook → copiar URL | Notificar al cliente cuando CocoLuxe asigna el vehículo |

> **GOOGLE MAPS API KEY:** *En Google Cloud Console habilitar: Maps JavaScript API. Restringir la key solo al dominio cocoluxepuntacana.com. Nunca dejarla sin restricción de dominio — cualquiera podría usarla y generar cargos.*

# 9. Checklist de Implementación — En Orden Estricto

No pasar al siguiente punto hasta que el anterior funciona correctamente.

### BASE DE DATOS

1. Crear tabla live_tracking con el SQL de la sección 5.1
2. Verificar que Realtime está activado en live_tracking
3. Verificar que RLS permite lectura pública solo cuando is_active = TRUE
4. Prueba: insertar un registro manualmente y verificar en Supabase Studio

### VARIABLES DE ENTORNO

5. NEXT_PUBLIC_GOOGLE_MAPS_KEY — Maps JavaScript API habilitada y restringida al dominio
6. MAKE_WEBHOOK_TRIP_COMPLETED — crear el escenario en Make primero
7. MAKE_WEBHOOK_FEEDBACK — crear el escenario en Make primero
8. MAKE_WEBHOOK_VEHICLE_ASSIGNED — crear el escenario en Make primero
9. Verificar que las 4 variables están en Vercel Production y en .env.local

### PÁGINAS Y COMPONENTES

10. Crear src/components/LiveMap.jsx con los estilos negro/dorado de la sección 5.2
11. Crear src/app/[locale]/reserva/[id]/page.jsx con la lógica condicional de la sección 3.1
12. Crear src/app/[locale]/seguimiento/[id]/page.jsx con la sección 5.3
13. Crear src/app/[locale]/feedback/[id]/page.jsx con las 3 preguntas en 4 idiomas
14. Crear src/app/api/trips/complete/route.js
15. Crear src/app/api/feedback/route.js con el código de la sección 7.2

### MAKE — 3 FLUJOS ADICIONALES

16. Flujo "Vehículo Asignado": Supabase UPDATE booking → Resend email + SMS al cliente
17. Flujo "Viaje Completado": Webhook trips/complete → Sleep 2h → Resend encuesta
18. Flujo "Encuesta Respondida": Webhook feedback → Supabase → HubSpot → Alerta si rating ≤ 2

### PRUEBAS COMPLETAS — En Este Orden

19. Crear una reserva de prueba en Supabase sin vehículo asignado
20. Abrir /reserva/[id] en el móvil — debe mostrar Estado 1 con mensaje de pendiente
21. Actualizar el campo vehicle y plate en Supabase — la página debe actualizarse sola en tiempo real
22. El cliente debe recibir SMS/email de "Tu vehículo está confirmado"
23. Tocar "Iniciar mi viaje" en el móvil — el navegador pide permiso de GPS
24. Aceptar el permiso — el mapa aparece con el punto dorado
25. Abrir /seguimiento/[id] en otro teléfono — debe ver el mismo punto moviéndose

26. Tocar "Llegué" — el mapa se congela, Make recibe el webhook
27. Cambiar el Sleep de Make a 1 minuto para probar — el email de encuesta debe llegar
28. Responder la encuesta con rating 2 — verificar que llega alerta a Kelmin
29. Responder la encuesta con rating 5 — verificar que HubSpot se actualiza

- [x] Volver a cambiar el Sleep a 7200 segundos (2 horas) antes del lanzamiento en producción

# 10. Resumen Final

| TECNOLOGÍA USADA | COSTO ADICIONAL TOTAL |
| --- | --- |
| ✅ navigator.geolocation — GPS del navegador<br>✅ Supabase Realtime — distribución en tiempo real<br>✅ Google Maps JavaScript API — mapa en vivo<br>✅ Make — automatización de notificaciones<br>✅ Resend — emails transaccionales<br>✅ /feedback/[id] — encuesta propia<br>❌ **HyperTrack — NO se usa en Fase 1** | **$0**<br>Todo usa servicios ya incluidosen el stack de CocoLuxe.<br>*Google Maps: ~$2 por cada 1,000 sesiones(centavos al mes en volumen inicial)* |

> **PRUEBA DE ACEPTACIÓN FINAL:** *Hacer una reserva real de $1 entre dos teléfonos. Verificar los 4 estados de la página. Confirmar que el seguimiento se ve en el segundo teléfono. Confirmar que el email de encuesta llega 2 horas después (o 1 minuto en modo prueba). Si todo funciona, el hito está completo.*